while developing machine learning or deep learning models we have situations in which

training accuracy ↑ but

Validation accuracy ↓

or

testing accuracy ↓

this is the case which is known as

Overfitting

→ Phenomenon where model doesn't perform well on unseen data. Because it learn the noise in traing data as well

↓

and tends to capture every points.

Here model memorizes the training data instead of learning the pattern in

Underfitting

→ The model lacks the capacity to grasp the data's complexity or training ends too easily.

→ It occurs when we choose a model that is too simplistic for complex real-world data.
(The model simply lacks the capacity to extract meaningful patterns)

(The Simplifies)

Bias :- It is the error introduced by simplifying assumptions made by a model to make the target function easier to learn.

→ Always deals with "Training Data"

Types :-

1) **Low Bias :-**

When model is sufficiently complex and can capture the true relationships b/w input and output

2) High Bias :-

"When the model is too simple, if Underfit the data"

- This means the model does not capture important patterns in data,
- Poor performance on both → Training Data, Testing Data.

(The overthinker)

Variance :- Refers to errors due to the model's sensitivity to small fluctuation in training data

always deals with "Testing data"

Types :-

1) Low Variance :- When the model is general enough to perform consistently across both the training and testing data.

2) High Variance :-

when the model is too Complex, it Overfits the data

- This means the model performance well on training set but poorly on new, Unseen testing data.
- Because it has memorized the training data rather learning general patterns.

Bias - Variance Tradeoff

We can't Minimize both bias and variance at same time.

↳ As we change our model to reduce one, the other naturally increases.

a) Increasing Model Complexity → { Bias (↓) Variance (↑) }

b) Decreasing Model Complexity → { Bias (↑) Variance (↓) }

B

The goal of Model is to find a middle ground - where total error is minimized, ensuring the model generalizes well to new, unseen data.

Errors in ML :-

1) Irreducible errors (always Present in Model (noise)
2) Reducible errors (Can be reduce to an extent)

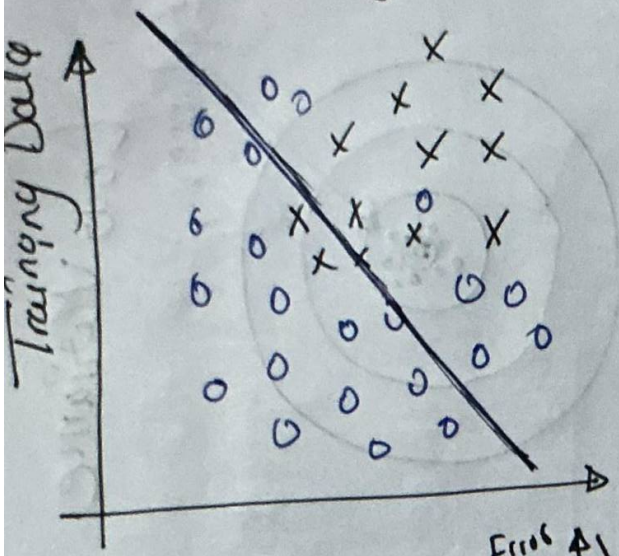
Bias Variance (Balance them is called bias-Variance tradeoff)

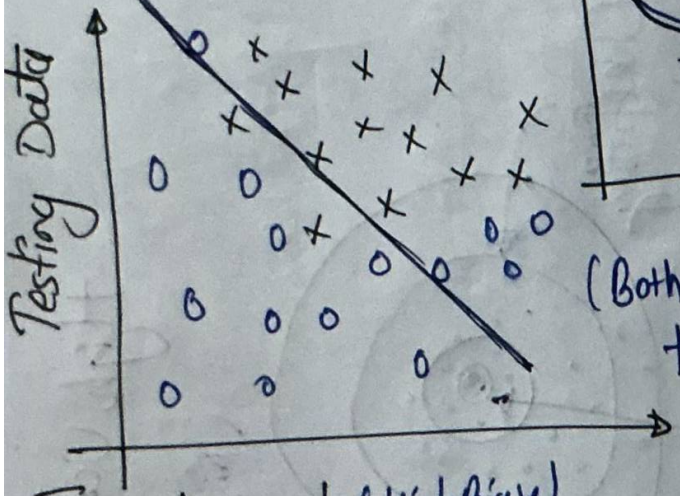
↓

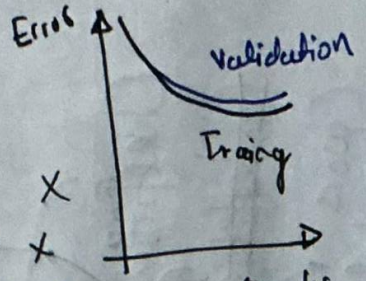
Goal To find : low bias & low Variance.

Irreducible Error (The Noise) : This is the inherent randomness in noise in data set that Machine learning algo can never eliminate or predict.

Underfitting



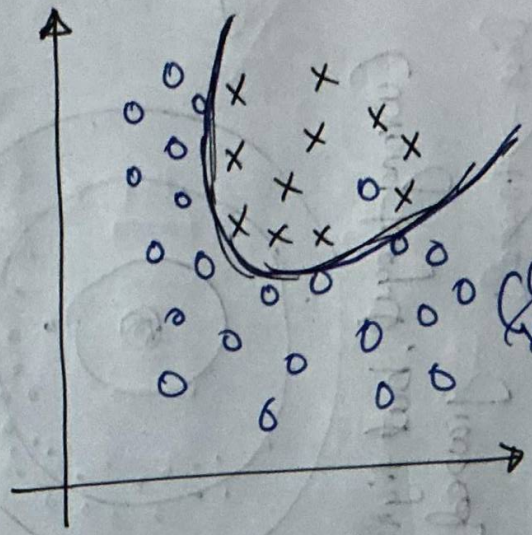




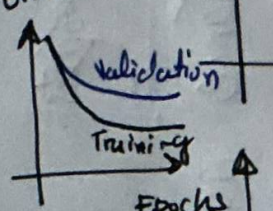
(Both Errors Stays High)

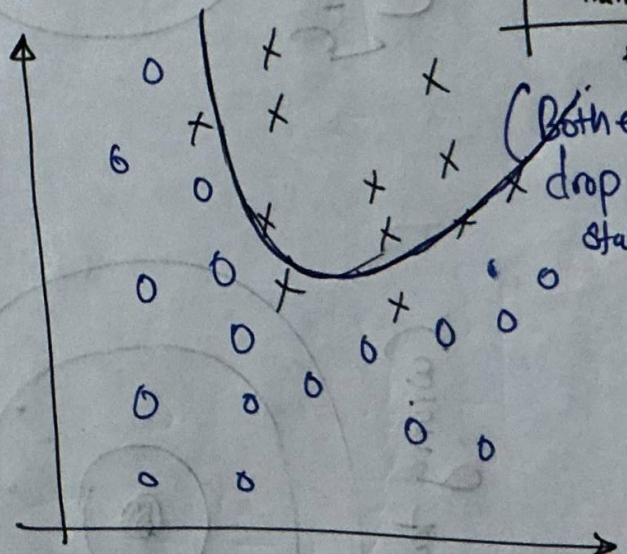
- Training Accuracy ↓ (High Biase)
- Testing Accuracy ↓ (High variance)

Optimum



(Gap wides val err goes up)



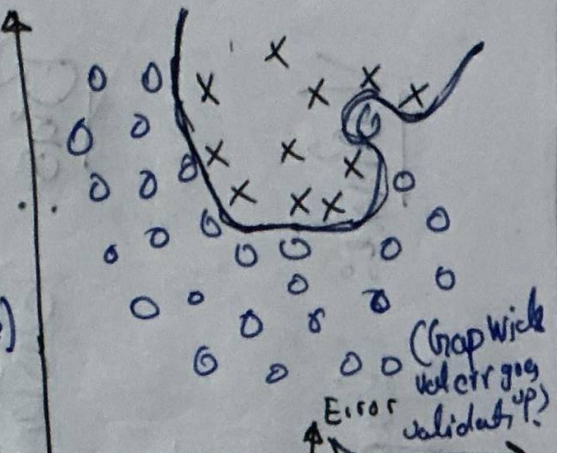


(Both errors drop and stabilize)

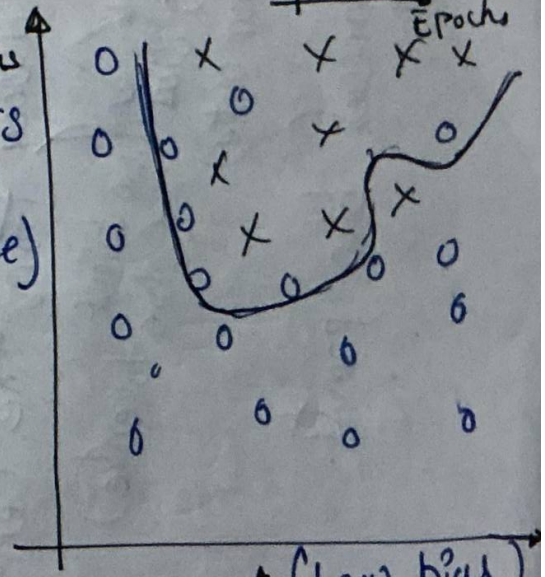
Training Accuracy ≈ Test Accuracy

(low Bias) & (low variance)

Overfitting



(Gap wide val err goes up)



Training Acc ↑ (Low bias)

Testing Acc ↓ (High Vari

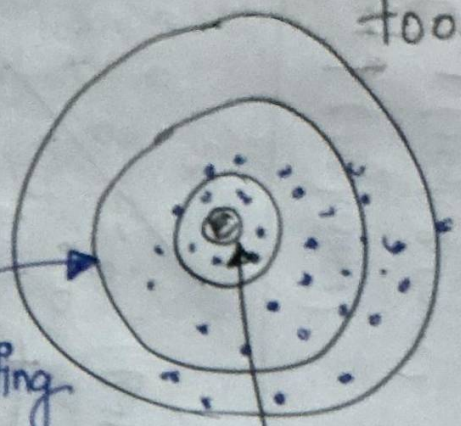
Low Variance

High Variance.

Low Bias

High Bias

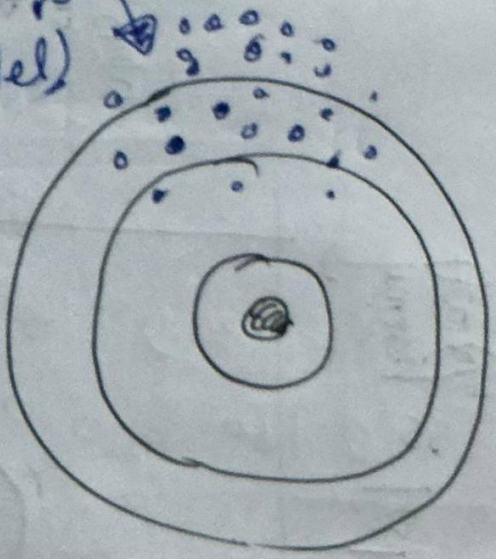
(Right target on avg, too Scattered.



~~Overfitting~~
Underfitting

(wrong target, but tightly grouped)

(wors Model)



(wrong target, wildly Scattered)

Bias → Avg dist from centre

Variance → Gap blw points.

Bias: Distance away from the centre

High Bias (far from centre)

Low bias (close to centre)

Variance: Gap/Spread blw points.

Bias Formula :- (Error blw avg model prediction and ground-truth)

$$\text{Bias} = \mathbb{E}[f'(x)] - f(x)$$

- $\mathbb{E}[f'(x)]$ → Expected (Avg) Prediction (if trained the model many times on different data!)
- $f(x)$ → The absolute true value (Ground-truth)

Variance

Formula :- (Avg variability in model prediction for given dataset)

$$\text{Variance} = \mathbb{E}\left[\left(f'(x) - \mathbb{E}[f'(x)] \right)^2 \right]$$

- $\mathbb{E}[f'(x)]$ → The avg prediction of all models combined
- $f'(x)$ → A specific individual prediction made by one trained model.
- ^{2} → Positive & penalizes for large gap

Total Error

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}.$$

If we force Bias down by making complex → Variance naturally shoots up and vice versa.

Eg. Imagine the absolute true value for a data point is $f(x) = 10$.

we train three variations of a model on different samples of data, they give us three individual predictions: 9, 11, and 13

Step: 1

$$\text{Average Prediction } (E[f(x)]) = \frac{9+11+13}{3} = 11$$

Step: 2

$$\text{Bias} = E[f(x)] - f(x)$$
$$= 11 - 10 = 1$$

$\boxed{\text{Bias}^2 = 1}$ ⇝ On avg our model overshoots the truth value by 1 unit. (Low Bias)

Step: 3

The formula looks at how much each individual prediction deviates from our Avg Prediction (11).

- For Prediction 9: $(9-11)^2 = 4$
- For Prediction 11: $(11-11) = 0$
- For Prediction 13: $(13-11) = 4$

$$\text{Variance} = \frac{4+0+4}{3} = \frac{8}{3} \approx 2.67$$

Variance = 2.67

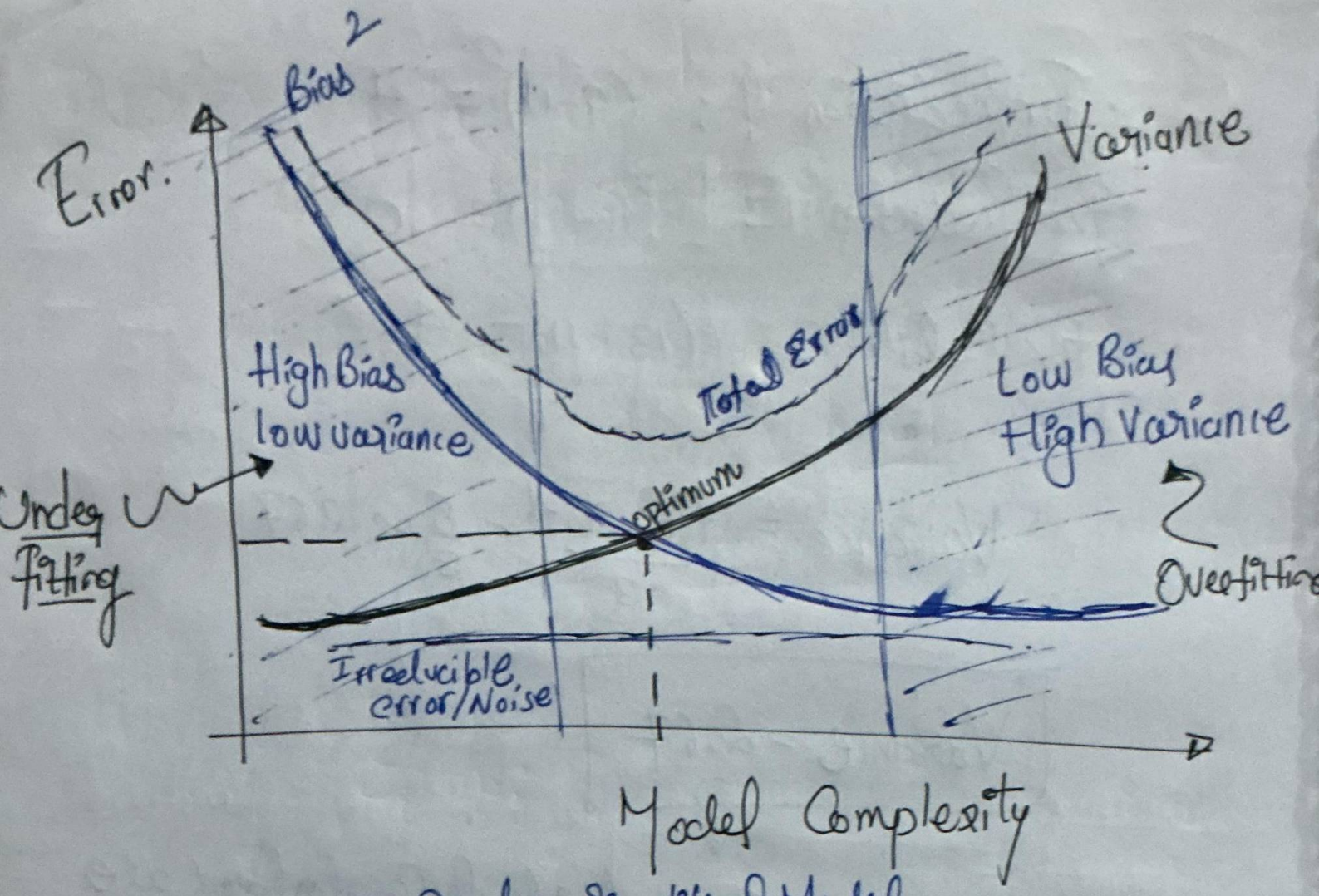
The individual predictions are moderately scattered around their own avg.

Step 4:-

$$\text{Total Error} = \text{Bias}^2 + \text{Variance}$$

$$= 1 + 2.67$$

Total Error = 3.67



High Bias
- Overly-Simplified Model
- Underfitting
- High Error on both test and training data

High Variance
- Overly Complex Model
- Overfitting
- Low Error on train data and high on test
- Starts Modelling the noise in the ~~output~~ input.

"The best Model is where the error is reduced.
Compromise b/w bias and variance"

Gold Note:

Methods to Reduce

(Countering underfitting requires adding Complexity, while countering Overfitting requires introducing constraints)

i) Underfitting

when a model is underfitting, it is too Simple to notice trends

↳ we must add More Complexity so it can learn.

a) Increase Model Complexity: Switch to inherently more Complex algorithms (e.g move from a Linear Regression to Random Forest or Polynomial regression)

b) Feature Engineering & Selection: create or inject more relevant feature, giving the model more Contextual data points to figure out relationships.

c) Decrease Regularization:

~~Give iterative algorithms more training time~~

If we are penalizing parameters too heavily (high L1/L2), dial them back so the model weights can adapt freely to data.

d) Train for More Epochs: Give iterative algorithms more training time so their weights have opportunity to converge and minimize error.

ii) Overfitting

when a model is overfitting it is memorizing data and random noise.

↓

we must restrict its freedom or expose it to broader patterns

a) Collect More training Data: Exposing the model to wider variety of unique examples makes it harder to memorize individual quirks.

b) Apply Regularization (L1/L2): Implement Ridge or Lasso penalties to append a mathematical restriction to the loss function, preventing any individual weight parameter from growing too large.

c) Implement Early Stopping: Monitor performance on a validation split and halt iterative training the exact moment validation error begins to rise.

d) Feature Pruning: Remove irrelevant, highly correlated, or noisy columns so the model focuses exclusively on core signals.

e) Use Dropout layers: While building deep neural networks, randomly deactivate a percentage of neurons during training batches to prevent co-dependency

iii) General Practices
(To Prevent Both Simultaneously)

By Utilizing two major architectural protocols
If ensure any model stays in
balanced "sweet spot"
— during development

a) Cross-Validation :
(eg. K-Fold)

"Never rely on a Single train-test Split."
Instead, rotate which chunks of data act as
training set Vs the validation set
across multiple trains loop.

- This yield a highly reliable, realistic
performance benchmark.

b) Ensemble
Methods: "Instead of relying on a
Single predictor, use
methods like

- Bagging (avg strong model to smoothly
reduce high variance/
overfitting)
- Boosting (Combines Sequential weak
Models to steadily reduce
high bias/underfitting)